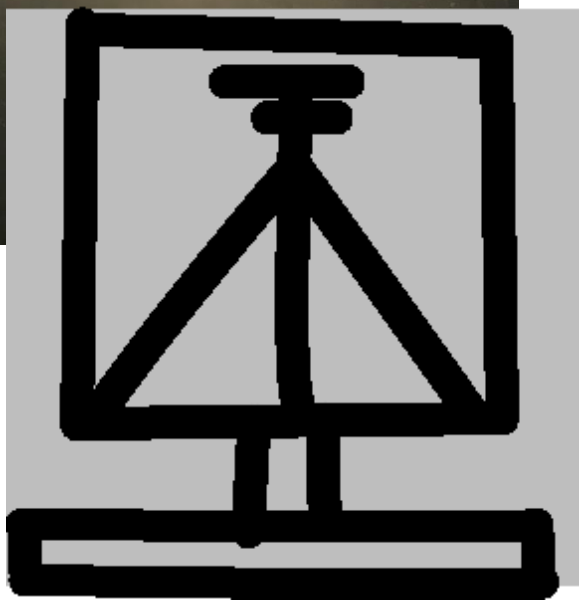
Module B4 "Géomatique + TI"

B4a "Interlis 1"





© CF-geo, C. Carneiro 08.2019

Table des matières

1. INTERLIS 4

1.1. Généralités 4

1.2. Historique d'INTERLIS 5

1.3. Concernant ce module 6

2. Langage de description 7

2.1. Exemple introductif d'une description INTERLIS 7

2.2. Syntaxe utilisée 8

2.2.1 Règles pour la notation de la syntaxe 8

2.3. Définition du langage de description 9

2.3.1 Symboles de base du langage 9

2.3.1.1. Noms 9

2.3.1.2. Nombres 10

2.3.1.3. Explication 10

2.3.1.4. Commentaire 10

2.3.1.5. Mots réservés 10

2.3.2 Structure principale du langage 11

2.3.2.1. Définition d'un transfert 11

2.3.2.2. La définition du domaine 11

2.3.2.3. Le modèle de données 13

2.3.2.4. Le thème 13

2.3.2.5. La table 13

2.3.2.6. L'attribut 14

2.3.3 Types d'attribut 16

2.3.3.1. Types de base 16

2.3.3.2. Concernant les coordonnées 17

2.3.3.3. Concernant l'énumération 17

2.3.3.4. Concernant la mise en place de textes 18

2.3.3.5. Types de lignes 20

2.3.3.6. Types de surfaces 21

3. Construction du fichier de transfert 24

3.1. Structure propre au système 24

3.1.1 Structure physique 24

3.1.2 Structure logique 24

3.2. Format libre et format fixe 24
3.2.1 Format libre 24
3.2.2 Format fixe 25
3.2.2.1. Caractère initial 25
3.2.2.2. Caractères suivants 25
3.3. Structuration selon le contenu 26
Exemple complet 27
4. Remarques concernant INTERLIS 29
4.1. Quelques règles 29
4.2. Concernant le style de description 29
5. Bibliographie et références 30
6. Exercices - Interlis 1 31
6.1. Re-engineering 31
6.2. Description du modèle « Parcelle / Pos_Numero » en modèle de données INTERLIS 1 32
6.3. Description du modèle « Batiment / Logement » en modèle de données INTERLIS 1 33
6.4. Exercice sur un jeu de données ITF 34
6.5. Description du modèle « Plan des zones » en modèle de données INTERLIS 136
6.6. Identification d'erreurs dans un jeu de données ITF 39

1. INTERLIS

1.1. Généralités

INTERLIS est un **langage de description** de données (Data Description Language DDL) pour des données à référence spatiale ou des « géodonnées ».

INTERLIS se rapproche du modèle relationnel avec extension à des types de données particuliers pour décrire des propriétés spatiales.

INTERLIS permet de définir des modèles conceptuels pour des bases de données à référence spatiale. Un modèle conceptuel décrit de manière neutre (indépendant d'un système concret) le contenu et la structure des données d'une application.

INTERLIS est un **langage formel** qui comprend un vocabulaire défini et une syntaxe. La syntaxe du langage suit les idées de langages de programmation tels que PASCAL ou MODULA2. Par conséquent une description en INTERLIS (« nom_de_fichier. ILI ») peut être traitée par un ordinateur. Pour pouvoir contrôler si un modèle décrit en INTERLIS est formellement correct, il existe un **compilateur**-INTERLIS qui protocole les éventuelles erreurs (« nom_de_fichier. LST »). En revanche les erreurs sémantiques (erreurs dans la modélisation) ne sont pas reconnues !

INTERLIS représente un « mécanisme » qui permet un **échange de données** entre systèmes d'informations à référence spatiale.

Si un modèle INTERLIS est correct, il est possible de créer un fichier qui décrit le format de transfert de données en fonction du modèle décrit. INTERLIS représente ainsi un « mécanisme » qui permet également un **échange de données** entre systèmes d'informations à référence spatiale (Figure 1).

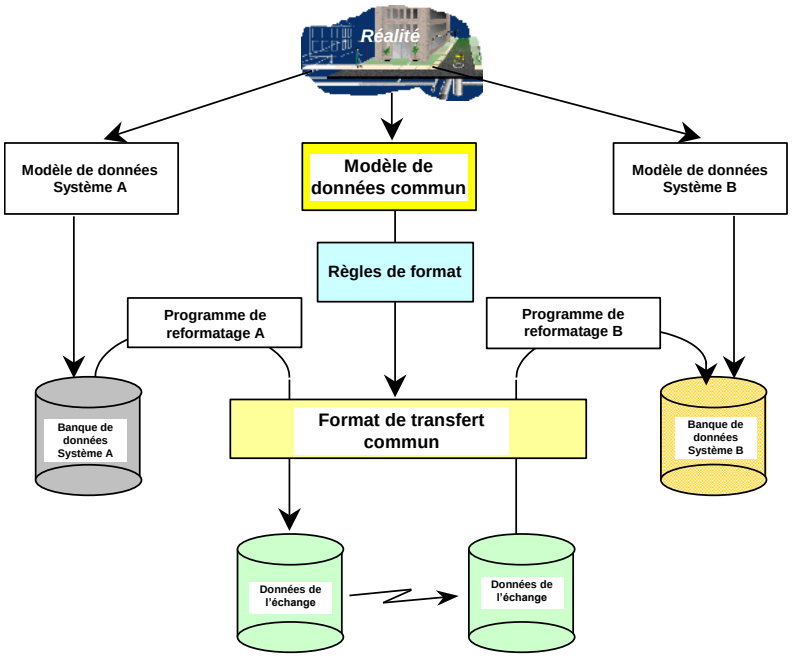


Figure 1 : Mécanisme d'échange de données (selon INTERLIS)

Pour pouvoir échanger des données sans perte d'informations entre SIG, il faut en premier lieu un modèle de données commun et ensuite la détermination du format d'échange.

Par conséquent les efforts doivent se concentrer dans un premier temps sur la définition et la création d'un **modèle de données** (modèle conceptuel) pour une application spécifique.

Si le format d'échange de données peut être automatiquement défini à partir du modèle de données (p.ex. description Interlis), on parle d'un **échange de données basé sur un modèle** (par rapport à un transfert uniquement **basé sur un format**, comme p.ex. DXF).

INTERLIS permet donc de décrire un modèle de données et de produire la description du format de transfert de données correspondante – mais INTERLIS ne produit pas de données !

1.2. Historique d'INTERLIS

Dans le cadre de la REMO (réforme de la mensuration officielle), la direction fédérale des mensurations cadastrales (D+M) a mandaté en 1988 un expert pour définir le processus d'échange de données cadastrales.

Résultat : pour pouvoir échanger des données entre différents systèmes et sans perte d'information, il faut tout d'abord décrire les données à échanger, et cela indépendamment d'un système concret. Un langage de description neutre est nécessaire - l'idée d'INTERLIS était née et continue d'évoluer jusqu'à nos jours (Tableau 1).

1991	INTERLIS Mécanisme d'échange de données pour systèmes d'information du territoire. 1ère version officielle.
1994	Ordonnance technique sur la mensuration officielle (OTEMO). INTERLIS est partie intégrante de l'interface de la mensuration officielle (IMO).
1998	Norme Suisse SN 612030 avec INTERLIS version 1.0
1998	Spécifications pour INTERLIS version 2.0
2001	INTERLIS version 2.0 prêt
2003	INTERLIS 2 Norme suisse (SN612031) (Version 2.2)
2006	INTERLIS 2 Norme suisse (SN612031 (Version 2.3)
2016	INTERLIS 2 – Version 2.4 : projet de norme eCH-0031
2017	INTERLIS 2 – Version 2.4 : norme Suisse eCH-0031 version définitive 2.0
2019	Proposition d'utiliser INTERLIS version 2.4 comme DDL pour la norme ISO 19152 (« Land Administration Domain Model »)

Tableau 1 : Historique d'Interlis

1.3. Concernant ce module

Ce module présente les éléments les plus importants du langage de description INTERLIS.

Pour le reste il est conseillé d'étudier le document SN 612030 « INTERLIS Langue de modélisation et méthode de transfert de données» SNV 2001.

2. Langage de description

2.1. Exemple introductif d'une description INTERLIS

Diagrammes (Figure 2) :

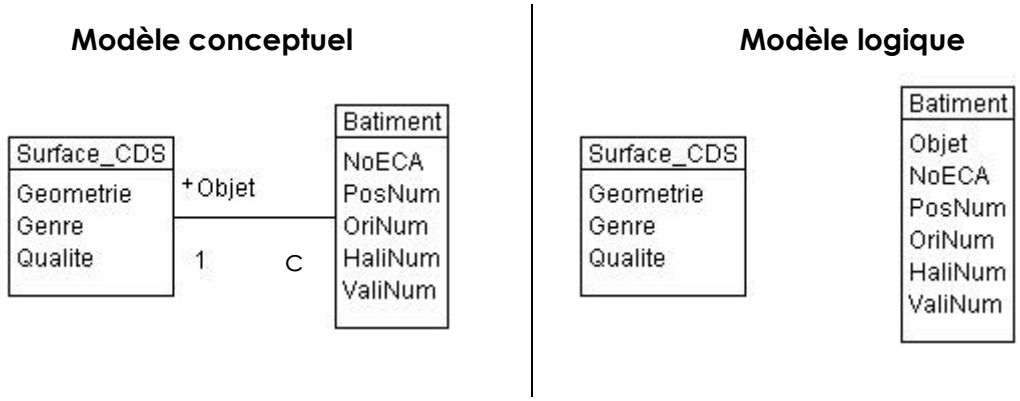


Figure 2 : Diagrammes (modèle conceptuel / modèle logique)

Description INTERLIS correspondante :

Description complète d'un modèle de données

Définition du modèle de données

Définition d'un thème du modèle

Définition d'une table d'un thème

Définition d'une table d'un thème

Définition format et codification

```
TOPIC Couverture_du_sol =

    TABLE Surface_CDS =
            Geometrie:   AREA WITH (STRAIGHTS, ARCS) VERTEX CoordN
                         WITHOUT OVERLAPS > 0.10;
            Genre:       (batiment, revetement_dur, verte, eau, boisee,
                         sans_vegetation);
            Qualite:     (MO93, NP);
    NO IDENT
    END Surface_CDS;

    TABLE Batiment =
            Objet:       -> Surface_CDS //Genre = batiment//; !!relation 1-c
            NoECA:       TEXT*6;
            PosNum:      CoordN //position dans la surface//;
            OriNum:      GRADS 0.0 399.9;
            HaliNum:     HALIGNMENT;
            ValiNum:     VALIGNMENT;
    IDENT
            NoECA;       !! NoECA est univoque
    END Batiment;

END Couverture_du_sol.
END Exemple.

FORMAT FREE;

CODE
        BLANK=DEFAULT, UNDEFINED=DEFAULT, CONTINUE=DEFAULT;
        TID=ANY
END.
```

2.2. Syntaxe utilisée

La syntaxe définit les règles de construction d'un langage à partir de mots ou de symboles et d'expressions.

Dans les chapitres suivants qui expliquent le langage INTERLIS, les signes particuliers et les mots réservés (mais pas la description des données) sont toujours écrits entre apostrophes (sauf dans les exemples concrets), p.ex. ',' ou 'MODEL'. Les mots réservés s'écrivent en majuscules.

2.2.1 Règles pour la notation de la syntaxe

Formalisme du langage : EBNF = Extended Backus Naur Formalism (formalisme étendu Backus Naur)

Formule dans le langage INTERLIS :

```
Nom de la formule = Expression de la formule
```

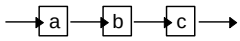
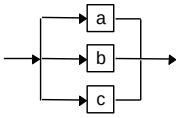
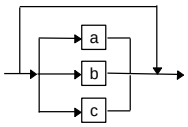
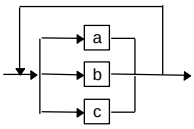
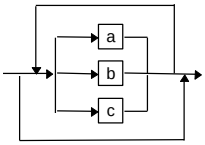
L'expression de la formule est une combinaison de :

- mots fixes (mots réservés, enfermés entre apostrophes ' ')
- signes spéciaux, p.ex. '='
- références d'autres formules par leur nom de formule.

Exemple :

```
Theme = 'TOPIC' Nom_du_theme'='
        (*Table | DefDomaine_local* )
        'END' Nom_du_theme '.'
```

Présentation des combinaisons possibles :

	EBNF	Diagramme de syntaxe	Description textuelle
Suite	a b c		d'abord a, puis b, puis c
Sélection obligatoire	(a | b | c)		a ou b ou c
Sélection facultative	[a | b | c]		a ou b ou c ou rien
Répétition obligatoire	(*a | b | c*)		Suite quelconque de a ou b ou c. Au moins une mention est exigée. « Rien » n'est pas possible
Répétition facultative	{a | b | c}		Suite quelconque de a ou b ou c ou rien. Ex. : aaabbcacc

Un diagramme de syntaxe représente par un graphisme les règles d'assemblage des expressions.

2.3. Définition du langage de description

2.3.1 Symboles de base du langage

Le langage de description INTERLIS distingue les classes de symboles suivantes : noms, nombres, explications, signes particuliers, mots réservés et commentaires.

2.3.1.1. Noms

Un nom est défini comme une suite de lettres, chiffres et traits de soulignement, où le premier caractère doit être une lettre. Les accents (é, à, etc.) ne sont pas autorisés.

```
Nom = Lettre {Lettre | Chiffre | '_'}
     Lettre = ('A' | .. | 'Z' | 'a' | .. | 'z')
     Chiffre = ('0' | '1' | .. | '9')
```



2.3.1.2. Nombres

Les nombres sont utilisés pour la définition de domaines et pour la codification d'un caractère donné.

```
NoPos = (*Chiffre*)
Nombre = ['+' | '-']NoPos
Dec = Nombre [ ' . ' NoPos ] [ Facteur_de_multiplication ]
Facteur_de_multiplication = ' S ' Nombre
```

2.3.1.3. Explication

Une explication décrit un fait avec plus de précision. Il n'y a (actuellement) aucune interprétation par le compilateur INTERLIS.

Une explication est enserrée entre deux doubles traits obliques et ne doit pas contenir de tels doubles traits !

```
//voici une explication//
```

2.3.1.4. Commentaire

Un commentaire permet de documenter une description INTERLIS avec des compléments nécessaires pour améliorer la compréhension. Le compilateur INTERLIS ignore les commentaires.

Deux points d'exclamation consécutifs ouvrent un commentaire. Celui-ci se termine à la fin de la ligne.

```
!!voici un commentaire
```

2.3.1.5. Mots réservés

Les mots suivants sont des mots réservés :

```
ANY             ARCS            AREA            BASE
BLANK           CODE            CONTINUE        CONTOUR
COORD2          COORD3          DATE            DEFAULT
DEGREES         DERIVATIVES     DIM1            DIM2
DOMAIN          END             FIX             FONT
FORMAT          FREE            GRADS           HALIGNMENT
I16             I32             IDENT           LAYOUT
LINEATTR        LINESIZE        MODEL           NO
OPTIONAL        OVERLAPPING     OVERLAPS        PERIPHERY
POLYLINE        RADIANS         STRAIGHTS       SURFACE
TABLE           TEXT            TID             TIDSIZE
TOPIC           TRANSFER        UNDEFINED       VALIGNMENT
VERTEX          VERTEXINFO      VIEW            WITH
WITHOUT
```

2.3.2 Structure principale du langage

2.3.2.1. Définition d'un transfert

```
TransferDef =   'TRANSFER' Nom_du_transfert';'
                [DefDomaine_global]
                Modele_de_donnees
                [Exploitations]
                {Vue}
                Format
                Codification
```

Un objectif principal du mécanisme INTERLIS est l'échange de données entre SIRS. C'est pour cette raison qu'une description INTERLIS commence avec le mot clé « TransferDef ».

Une définition de transfert se compose tout d'abord :

- d'un nom « Nom_du_transfert », puis
- de la définition facultative des domaines valables pour toute la définition (pour le modèle entier) « DefDomaine_global ».
- de la description proprement dite du modèle de données « Modèle_de_donnes »
- d'exploitations « Exploitations »et de vues « Vue » des données d'un modèle (les deux facultatifs)
- de la fixation des paramètres de transfert (format « Format » et codification « Codification » des données).

2.3.2.2. La définition du domaine

```
DefDomaine =    'DOMAIN'
                (*Nom_du_domaine '='Attribut_type';'*)
```

La définition d'un domaine sert à fixer un certain type de donnée avec toutes ses caractéristiques (p.ex. limites inférieures et supérieures dans le cas de coordonnées ou valeurs possibles d'un attribut, etc.).

Une telle définition peut être utilisée :

- dans l'entier de la définition de transfert, lorsque la définition des domaines intervient dans le cadre d'une définition de transfert
- à l'intérieur d'un modèle de données, si la définition du domaine intervient dans le cadre de la définition du modèle
- à l'intérieur d'un thème, si la définition du domaine intervient dans le cadre de la définition du thème.

Exemples :

```
TRANSFER Test
DOMAIN
       CoordN = COORD2      480000.00     60000.00
                            850000.00   320000.00;

MODEL exemple
```

Dans l'exemple introductif (chap 2.1), un nom de domaine « CoordN » (probablement l'abréviation pour « coordonnées nationales ») est défini avec le type d'attribut « COORD2 » (coordonnées en 2 dimensions) et des coordonnées qui fixent les point extrêmes d'un rectangle. Cette définition est valable dans l'entier de la description de transfert.

Signification de ce domaine : dès qu'on utilise le domaine « CoordN » pour un attribut dans la description, on définit une paire de coordonnées dans le système de coordonnées nationales, en « m » et avec une précision (ou peut-être mieux 'résolution') du « cm » et à l'intérieur du 'rectangle' !

Définition d'une table d'un thème

```
TOPIC Couverture_du_sol =

TABLE Surface_CDS =
          Geometrie:   AREA WITH (STRAIGHTS, ARCS) VERTEX CoordN
                       WITHOUT OVERLAPS > 0.10;
          Genre:       (batiment, revetement_dur, verte, eau, boisee,
                       sans_vegetation);
          Qualite:     (MO93, NP);
NO IDENT
END Surface_CDS;
```

Dans le thème « Couverture_du_sol » (exemple ci-dessus), l'attribut « Genre » peut avoir les valeurs « batiment, revetement_dur, verte, eau, boisee, sans_vegetation ».

Au lieu de définir ces valeurs auprès de l'attribut, on aurait pu introduire un domaine (voir exemple ci-dessous), qui serait alors une définition uniquement valable à l'intérieur du thème concerné.

Il suffit ensuite d'utiliser le nom de domaine « GenreCDS » auprès d'un attribut pour indiquer les valeurs admises.

```
TOPIC Couverture_du_sol =

DOMAIN
  GenreCDS =      (batiment, revetement_dur, verte, eau, boisee,
                  sans_vegetation);
```

Définition d'une table d'un thème

```
TABLE Surface_CDS =
          Geometrie:   AREA WITH (STRAIGHTS, ARCS) VERTEX CoordN
                       WITHOUT OVERLAPS > 0.10;
          Genre:       GenreCDS;
          Qualite:     (MO93, NP);
NO IDENT
END Surface_CDS;
```

2.3.2.3. Le modèle de données

```
Modele_de_donnees =     'MODEL' Nom_du_modele
                        [DefDomaine_modele]
                        (*Theme*)
                        'END' Nom_du_modele '.'
```

Le modèle de données est décrit par :

- un nom du modèle de données
- les domaines qui sont valides à l'intérieur du modèle
- les définitions des différents thèmes.

2.3.2.4. Le thème

```
Theme =         'TOPIC' Nom_du_theme '='
                (*Table | DefDomaine_local*)
                'END' Nom_du_theme '.'
```

Un thème est un ensemble de tables. Chaque thème est indépendant de l'autre.

2.3.2.5. La table

```
Table =           ['OPTIONAL']'TABLE' Nom_de_la_Table '='
                  Attributs Identifications
                  'END' Nom_de_la_Table ';'


Identifications = ('NO''IDENT' | 'IDENT' (*Identdef*))

Identdef =        Nom_attribut{','Nom_attribut}';'
```

Une table est un ensemble d'objets ayant les mêmes caractéristiques. Les objets sont décrits au moyen d'attributs et d'identifications.

Les attributs décrivent la structure exacte des données des objets.

Avec chaque définition d'une identification on définit un attribut ou une combinaison d'attributs qui identifie un objet de manière univoque.

« OPTIONAL » indique que la table ne doit pas obligatoirement exister ; elle est optionnelle.

Exemple 1 :

```
TABLE PFP3 =
  Numero: TEXT*12;
  Geometrie: CoordP ;
  PrecPlan: Precision;
  FiabPlan: Fiabilite;
  Signe: (borne, cheville_tuyau_pieu, non_materialise);
  Fiche: OPTIONAL (oui, non);       !! par défaut : non
  Provenance: OPTIONAL TEXT * 30;
IDENT
  Numero;
  Geometrie;
END PFP3;
```

Le numéro d'un objet doit être univoque dans une table PFP3. La géométrie d'un objet doit être univoque dans une table PFP3 (pas plusieurs points avec les mêmes coordonnées).

Exemple 2 :

```
  .....
  Provenance: OPTIONAL TEXT * 30;
IDENT
  Numero, Geometrie;
END PFP3;
```

La combinaison des deux attributs d'un objet de la table PFP3 doit être univoque (« numéro » ou « géométrie » seuls ne sont pas univoques).

2.3.2.6. L'attribut

```
Attributs =  (*Nom_attribut':'
             ['OPTIONAL'](Attribut_local | Attribut_relationnel)
             [Condition_de_coherence]';'*)

Condition_de_coherence =  Explication
```

Chaque attribut est décrit par son nom.

« OPTIONAL » indique que l'attribut ne doit pas obligatoirement exister (être saisi dans le cas concret), il est optionnel.

Les attributs sont classés en deux groupes :

Attributs relationnels :

Ces attributs indiquent les relations entre les objets des différentes tables. Pour assurer l'indépendance des couches de données, on ne peut utiliser les attributs relationnels qu'à l'intérieur d'un même thème !

```
Attribut_relationnel =    '->'Nom_de_la_table
```



Attributs locaux :

Un attribut local est un attribut d'un objet d'une table, qui n'est pas un attribut relationnel.

Pour décrire les caractéristiques d'un attribut, différents attributs-types (types de base et types géométriques) existent :

```
Attribut_local =          Domaine

Domaine =                 (Nom_du_domaine | Attribut_type);

Attribut_type =           (Type_de_base | Ligne | SurfLot)
```

Exemple :

```
TABLE Point_limite =
  Origine: -> Mise_a_jour_Fonds;       !! liaison 1-mc
  Identification: TEXT*12;
  Geometrie: CoordP;
  Provenance: OPTIONAL TEXT*30;
  Valeur: OPTIONAL [1..9];
IDENT
  Identification;
  Geometrie;
END Point_limite;
```

2.3.3 Types d'attribut

2.3.3.1. Types de base

Syntaxe	Exemples :
Coord2 = 'COORD2' ymin-Dec xmin-Dec ymax-Dec xmax-Dec	PosNum: COORD2 491000.000 116000.000 587000.000 206000.000 ;
Coord3 = 'COORD3' ymin-Dec xmin-Dec zmin-Dec ymax-Dec xmax-Dec zmax-Dec	PosNum: COORD3 491000.000 116000.000 200.000 587000.000 206000.000 2500.000;
Longueur = 'DIM1'min-Dec max-Dec	Altitude: DIM1 -200.00 5000.00;
Superficie = 'DIM2'min-Dec max-Dec	SurfBati: DIM2 10 500;
Angle = ('RADIANS' | 'GRADS' | 'DEGREES') min-Dec max-Dec	OriEcriture: OPTIONAL DEGREES 0.0 359.9;
Domaine_numerique = '['min-Dec '..'max-Dec']'	Valeur: [0..12];
Date = 'DATE'	Mise_a_jour: DATE;
Enumeration = '('Element{','Element}')' Element = Nom_element [Enumeration_subordonnee]	Materiau: (inconnu, beton (arme, precontraint), ciment);
Alignement_horizontal = 'HALIGNMENT'	HAliNum: HALIGNMENT !! défaut = centre
Alignement_vertical = 'VALIGNMENT'	VAliNum: VALIGNMENT !! défaut = milieu

Tableau 2 : Types de base pour des attributs

Remarque importante :

L'indication de minimum et maximum (p.ex. ymin-Dec xmin-Dec) pour les différents types de base définit non seulement les limites (p.ex. rectangle par deux coordonnées), mais aussi la « précision » ou la « résolution ». L'unité de mesure par défaut est le mètre.

Exemple :

```
DIM1 0.000 1000.000
```

Attribut de type longueur, valeur admise entre 0 et 1km, résolution du mm.

2.3.3.2. Concernant les coordonnées

Avec « COORD2 » et « COORD3 » on définit un domaine (avec un nom de domaine, p.ex. « CoordP ») pour les valeurs Y,X d'un point en 2 dimensions (Figure 3) et pour les valeurs Y,X,Z d'un point en 3 dimensions respectivement.

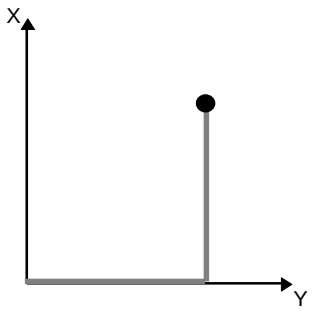


Figure 3 : Représentation d'un point en 2 dimensions

Y est la coordonnée positive vers l'EST et X vers le NORD (système de coordonnées géodésique).

2.3.3.3. Concernant l'énumération

Une énumération fixe un ensemble de valeurs définies. Une énumération peut être linéaire ou arborescente. Une subdivision est définie par des parenthèses. Syntaxe voir chapitre « types de base ».

Exemple d'une énumération linéaire :

```
(Batiment, Revetement_dur, Verte, Eau, Boisee, Sans_vegetation);
```

Exemple d'une énumération arborescente (Figure 4) :

```
Genre : (
      Batiment (
            habitat,
            agriculture,
            services,
            commerce,
            restauration,
            industrie,
            autre_batiment),
      Revetement_dur (
            route_chemin,
            trottoir,
            ilot,
            chemin_de_fer,
            place_aviation,
            bassin,
            autre_revetement_dur,
            acces_place),
      Verte (
            champ_pre_paturage,
            culture_intensive (
                  vigne, autre_culture_intensive),
            jardin_aisance,
            tourbiere,
            autre_surface_verte,
```

```
            paturage_estivage,
            vigne,
            verger,
            marais),
      Eau (
            cours_eau,
            eaux_stagnantes,
            roseliere),
      Boisee (
            foret_dense,
            autre_boisee,
            paturage_boise,
            tourbiere_boisee),
      Sans_vegetation (
            rocher,
            glacier_neve,
            eboulis_sable,
            graviere_decharge,
            autre_sans_vegetation,
            carriere_graviere,
            decharge)
);
```

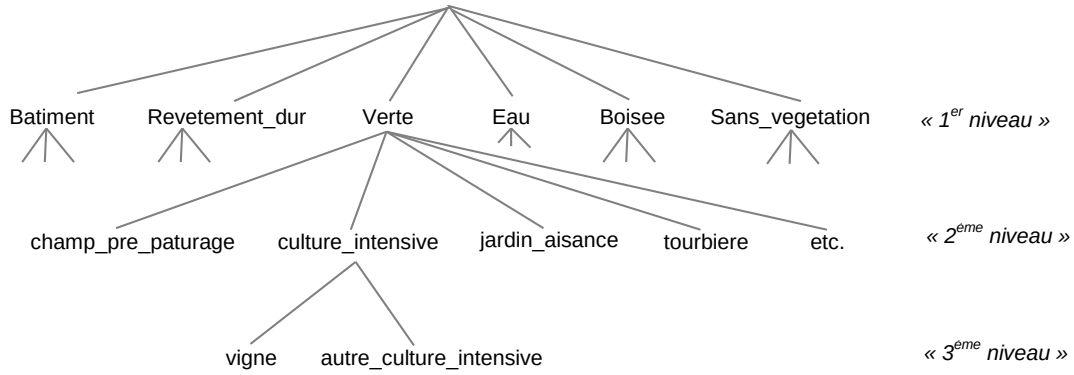


Figure 4 : Enumération pour genres de couverture du sol

2.3.3.4. Concernant la mise en place de textes

Pour la visualisation des données et pour l'édition de plans, il faut définir la position des textes. L'alignement horizontal et vertical permet de définir exactement le point par rapport auquel le texte se positionne. Syntaxe voir chapitre « types de base » (Figure 5).

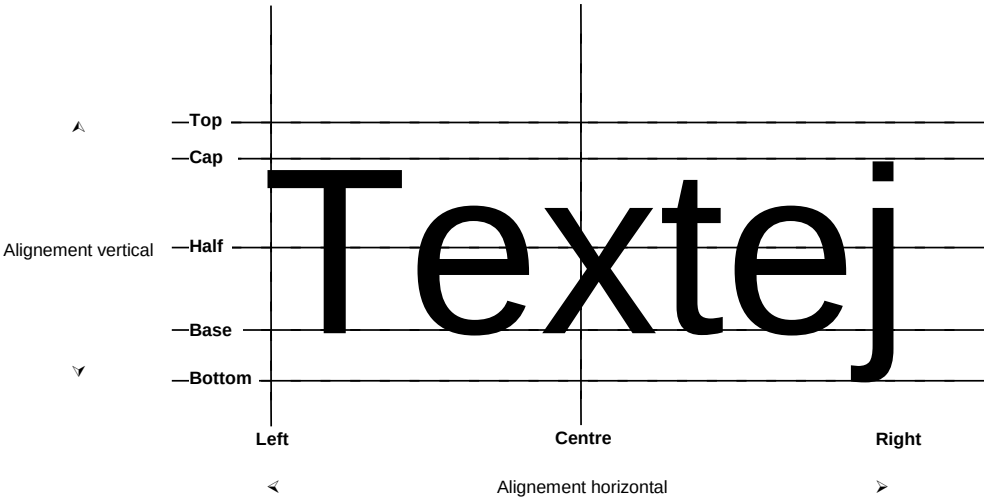


Figure 5 : Mise en place de textes

Les mots clés « HALIGNEMENT » et « VALIGNEMENT » sont des énumérations prédéfinies, avec les valeurs suivantes :

```
HALIGNMENT = (Left, Center, Right)
VALIGNMENT = (Top, Cap, Half, Base, Bottom)
```

Exemple de positionnement de texte et conséquence en cas de changement d'échelle (Figure 6 et Figure 7).

Ci-dessous : Halignment = Centre, Valignment = Half

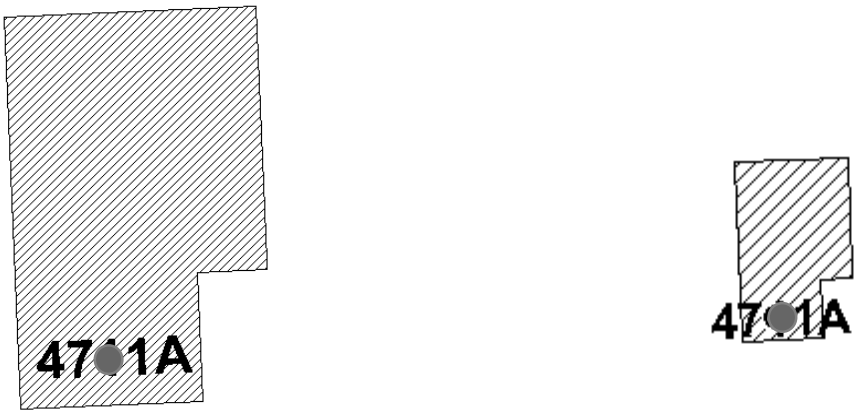


Figure 6 : Exemple de positionnement de texte

Ci-dessous : Halignment = Left, Valignment = Base

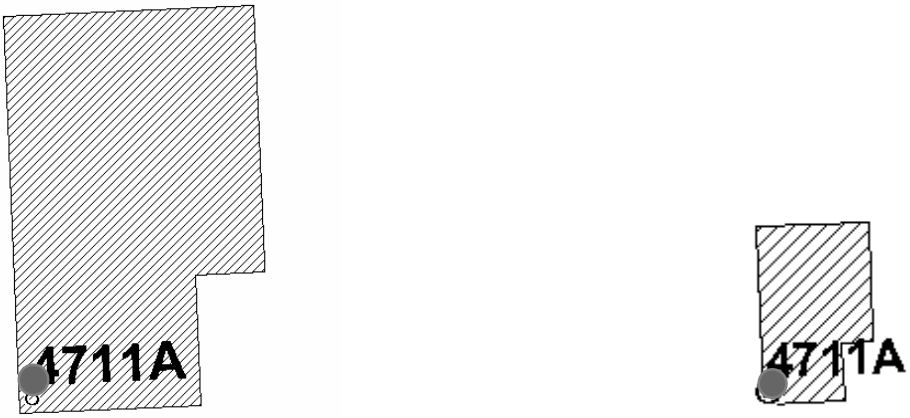


Figure 7 : Exemple de positionnement de texte

2.3.3.5. Types de lignes

Une ligne est une suite continue de segments de ligne, une polyligne.

```
Ligne =          'POLYLINE' Forme Points_appui [Intersec]

Forme =          'WITH' '('Type_Forme {','Type_Forme}')'

Type_Forme =     ('STRAIGHTS' | 'ARCS'| Explication)

Intersec =       'WITHOUT' 'OVERLAPS' '>' Dec

Points_appui =   'VERTEX' (Coord2 | Coord3 | Nom_du_type_de_coord)
                 ['BASE' Explication]
```

Exemple :

```
BordMur : POLYLINE WITH (STRAIGHTS, ARCS) VERTEX CoordP;
```

Formes géométriques possibles : combinaison de droites (« STRAIGHTS »), d'arcs de cercle (« ARCS ») et formes spéciales, à décrire avec une « explication » (Figure 8).

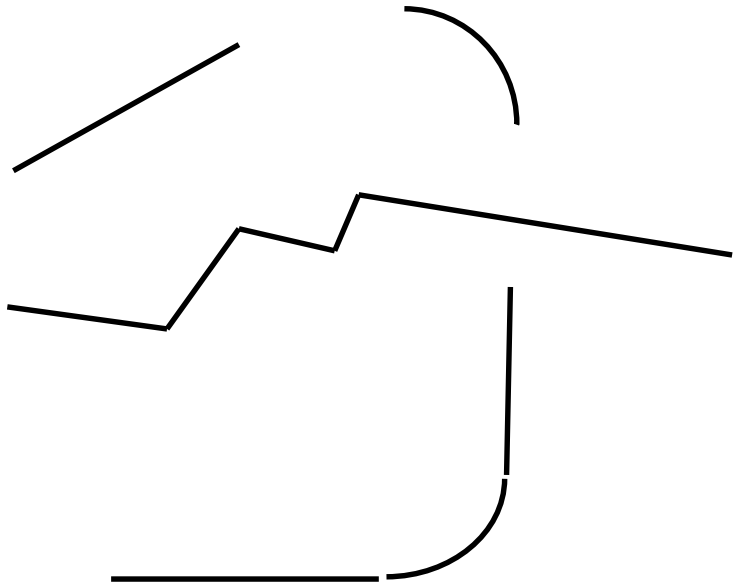


Figure 8 : Exemples de formes géométriques possibles

Il faut donner les caractéristiques des points d'appui (dans l'exemple ci-dessus : « CoordP »). Sont appelés points d'appui les points de début et de fin d'une polyligne, ainsi que les points à l'intérieur d'une polyligne.

Signification du terme « Intersec » : on peut demander qu'une ligne ne se coupe ni avec elle-même ni avec une ligne de même attribut mais d'un autre objet.

Ainsi on peut définir une tolérance (dans les mêmes unités que les coordonnées des points d'appui) pour des recoupements comme dans l'exemple ci-dessous. De tels recoupements (quelques mm ou cm) sont inévitables dans certains cas pour des raisons numériques (cercle tangent) (Figure 9).

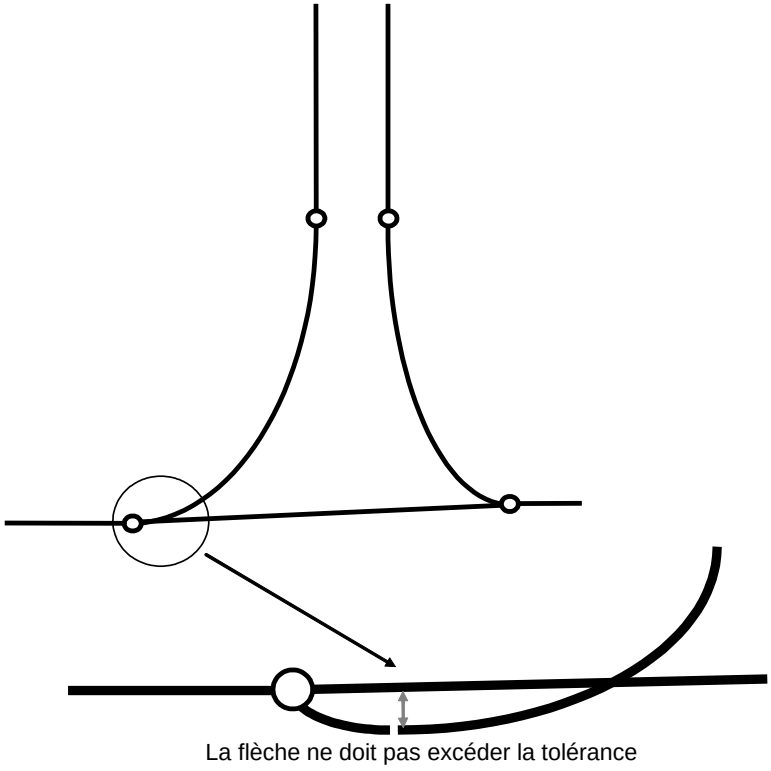


Figure 9 : Tolérance de « overlaps »

Exemple :

```
BordTrottoir : POLYLINE WITH (STRAIGHTS, ARCS) VERTEX CoordP
WITHOUT OVERLAPS > 0.4;
```

2.3.3.6. Types de surfaces

INTERLIS distingue entre surfaces (SURFACES) et lots (AREA) (Figure 10 et Figure 11). Les lots sont une forme particulière de surfaces sans recouvrement (partition du territoire).

```
SurfLot =       ('SURFACE' Forme Points_appui [Intersec] |
                 'AREA' Forme Points_appui Intersec)
                 [Lineattrdef]
```

```
Lineattrdef =     'LINEATTR' '='Attributs [Identifications] 'END'
```

Type SURFACE :

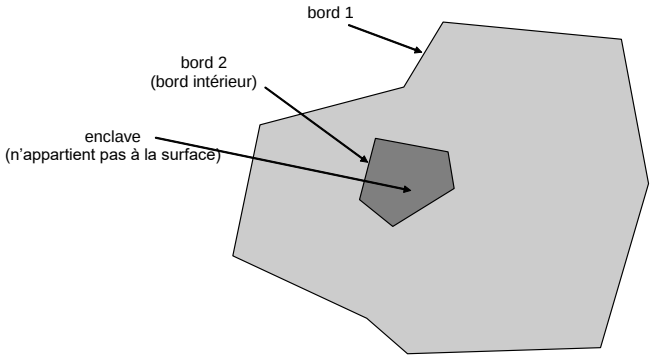


Figure 10 : Surface de type « SURFACE »

Des îlots sont admis (une surface peut avoir un ou plusieurs bords intérieurs (enclaves)).

Un recoupement entre plusieurs surfaces est permis.

Un bord peut être divisé en plusieurs polylignes.

Au maximum un attribut de type SURFACE par table.

Type AREA :

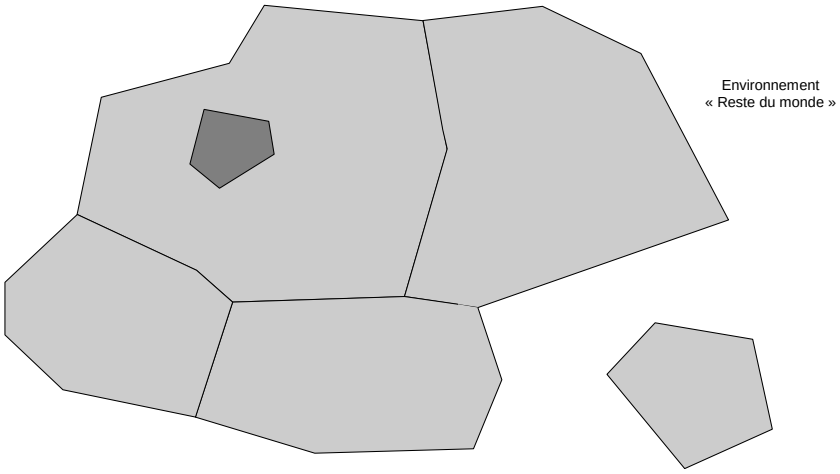


Figure 11 : Surface de type « AREA »

Des îlots sont admis (une surface de type AREA peut avoir un ou plusieurs bords intérieurs (enclaves)).

Un recoupement entre surfaces n'est pas permis.

Un bord peut être divisé en plusieurs polylignes.

Au maximum un attribut de type AREA par table.

Exemple :

```
TABLE Bien_Fonds =
  Geometrie: AREA WITH (STRAIGHTS, ARCS) VERTEX CoordP BASE
    // CoordP uniquement PFP, PL et PL territoriale //
    WITHOUT OVERLAPS > 0.050
    LINEATTR =
      Type: OPTIONAL (normal, mitoyen);  !! spécificité cantonale
    END;
  Superficie: DIM2 1 999999999;
NO IDENT
END Bien_Fonds;
```

3. Construction du fichier de transfert

3.1. Structure propre au système

Un fichier de transfert - indépendamment de son contenu - se compose d'une suite de lignes logiques. Dans les descriptions qui suivent, la fin d'une ligne sera toujours notée NL. La construction d'une ligne logique peut se considérer de deux points de vue : physique et logique.

3.1.1 Structure physique

Une ligne logique peut se décomposer en un nombre quelconque de lignes physiques. Le début de la première et la fin de la dernière ligne physique ne nécessitent aucune mention particulière. La fin d'une ligne physique qui ne correspond pas à la fin d'une ligne logique doit comporter le caractère "suite" (habituellement on utilise \). Avec la description de la codification (cf. section 2.2.10. du manuel de référence d'Interlis 1), on peut définir un autre caractère. Au début de la ligne "suite", on introduit la marque CONT comme reconnaissance de ligne et un caractère blanc. Les caractères avant le caractère suite appartiennent à la ligne logique, et les caractères après le caractère blanc à la ligne suivante, à l'exclusion du caractère suite, du caractère CONT et du caractère blanc.

INTERLIS ne règle pas la façon dont le fichier de transfert est décomposé en lignes physiques. Ceci est l'affaire d'un niveau interface plus bas. Sans convention particulière, les lignes physiques sont fermées par LF (Line Feed, ASCII OxA). Immédiatement avant LF, on peut trouver un caractère facultatif CR (ASCII OxD).

3.1.2 Structure logique

La ligne logique comprend une suite de champs. L'ordonnancement des champs dans la ligne et la relation avec la structure physique est dépendant de la façon de formater la partie "données" du fichier de transfert (cf. section 2.2.9. du manuel de référence d'Interlis 1).

La description ci-après de la structuration des données et de la codification de chaque champ ne se réfère qu'à la composition logique des lignes. La question de la séparation des champs entre eux dépend du choix du format fixe ou libre (cf. section 3.2. du manuel de référence d'Interlis 1). Dans les deux cas, il faut au minimum un caractère de séparation (blanc ou tabulateur) entre deux champs d'une même ligne logique.

3.2. Format libre et format fixe

La partie "données" d'un fichier de transfert peut avoir un format libre ou fixe. Le format libre est bien adapté aux langages de programmation modernes comme PASCAL ou MODULA, alors que le format fixe convient mieux aux langages tels le BASIC et le FORTRAN.

3.2.1 Format libre

Dans le format libre, les champs sont séparés au moins par un caractère blanc ou un tabulateur. Les longueurs des champs sont égales aux longueurs des valeurs actuelles. Une ligne logique peut être interrompue à n'importe quel endroit, même au milieu d'un champ (cf. section 3.1. du manuel de référence d'Interlis 1). Les caractères nécessaires à la

séparation doivent être simplement éliminés lors du décodage, pour rétablir la suite des caractères de la ligne logique.

S'il n'y a que des caractères indéfinis sur le reste d'une ligne logique, il est possible, dans le format libre, de mettre à leur place deux caractères indéfinis successifs et de fermer la ligne logique.

3.2.2 Format fixe

Dans le format fixe, les champs doivent être séparés par un seul caractère blanc. Chaque champ a une forme spécifique.

3.2.2.1. Caractère initial

(si l'attribut correspondant est facultatif) Ce caractère indique si la valeur est définie ou non. Si la valeur est définie, le caractère est blanc. Si la valeur n'est pas définie, on utilise le caractère non-défini (cf. section 3.4.4. du manuel de référence d'Interlis 1).

3.2.2.2. Caractères suivants

n caractères suivants pour la valeur. Pour un champ déterminé, il faut réserver le plus grand nombre de caractère qu'il est nécessaire pour la valeur en question (longueur maximum). Les définitions concrètes se trouvent au chap. 3.4. Les champs numériques sont toujours alignés à droite, les champs alphanumériques à gauche. Les caractères vides à la fin d'un champ restent des caractères vides, tandis qu'au début ou à l'intérieur d'un champ alphanumérique, ils sont occupés par le signe de remplacement (cf. sections 3.4.5. et 2.2.10. du manuel de référence d'Interlis 1).

Dans le cas où tous les champs restants n'ont pas suffisamment de place dans le cadre de la longueur déterminée de la ligne pour le format fixe (cf. section 2.2.9. du manuel de référence d'Interlis 1), la ligne doit être subdivisée. Cette subdivision se fait de telle sorte qu'un champ donné puisse être entièrement compris dans une ligne. Une ligne "suite" ne commencera que lorsque le champ actuel - pour autant qu'il ne s'agisse pas du dernier champ d'une ligne logique - et les caractères de séparation (caractères vides) et de suite, n'auront plus la place suffisante sur la ligne actuelle. Les caractères de séparation et de suite doivent donc toujours se trouver sur l'ancienne ligne.

Le format fixe est donc ainsi fait qu'il puisse également être lu selon les règles du format libre.

Pour tenir compte de la lecture en format fixe, la succession des lignes et des marques de lignes a été choisie de telle sorte que la prochaine ligne puisse être lue avec le format prédéterminé. Une relecture (Reread) n'est donc pas nécessaire. Dans de nombreux cas, plusieurs lignes se suivent avec la même marque de ligne. La fin de la séquence est formée d'une ligne qui ne contient qu'une seule marque de ligne typique de fin de la séquence et on teste si la marque de ligne n'indique pas la fin de la séquence. Dans le cas des figures géométriques, il faut tenir compte en sus que le même format peut se rencontrer toutefois avec des marques de lignes différentes pour des formes géométriques différentes.

3.3. Structuration selon le contenu

```
Fichier_de_transfert = 'SCNT' NL Description_du_contenu
                       ('MOTR' NL Description_du_modele_et_du_transfert)
                       |'MTID' Nom-transfert NL)
                       Donnees
                       'ENDE' NL.
Description = { Ligne_de_description } '////' NL .
Ligne_de_description = quelconque_sauf_////_au_début_de_ligne NL.
Donnees = { ModeleD }.
ModeleD = 'MODL' Nom-du_modele NL
          { ThemeD }
          'EMOD' NL.
```

Un fichier de transfert contient tout d'abord une indication sur le contenu, notamment le secteur géographique. Cette indication est définie comme pur texte sans formalisation. Les partenaires intéressés au transfert peuvent eux-mêmes définir une formalisation. Puis intervient la description du transfert (cf. section 2.2. du manuel de référence d'Interlis 1) respectivement un avis, qui identifie d'une manière univoque le transfert reconnu par chaque partie. Enfin suivent les données. Celles-ci sont organisées par niveau, dans la suite de la définition du modèle. Il est admissible que tous les thèmes du modèle ne soient pas transférés. Si un thème l'est, il l'est complètement (Figure 12).

```
ThemeD = 'TOPI' Nom-du_theme NL
         { TablesD }
         'ETOP'.
TablesD = 'TABL' Nom-de_la_table NL
          { ObjetD }
          [ Perimetre ]
          'ETAB' NL .
ObjetD = 'OBJE' Texte-TransferID AttributD .
AttributD = { Attribut_de_base |Coordonnee-de_ref_lot } NL
            { Attribut_de_ligne } { Asurface|Alot }   { TablesA }
[ Perimetre ] = 'PERI' Texte-TransferID  NL.
```

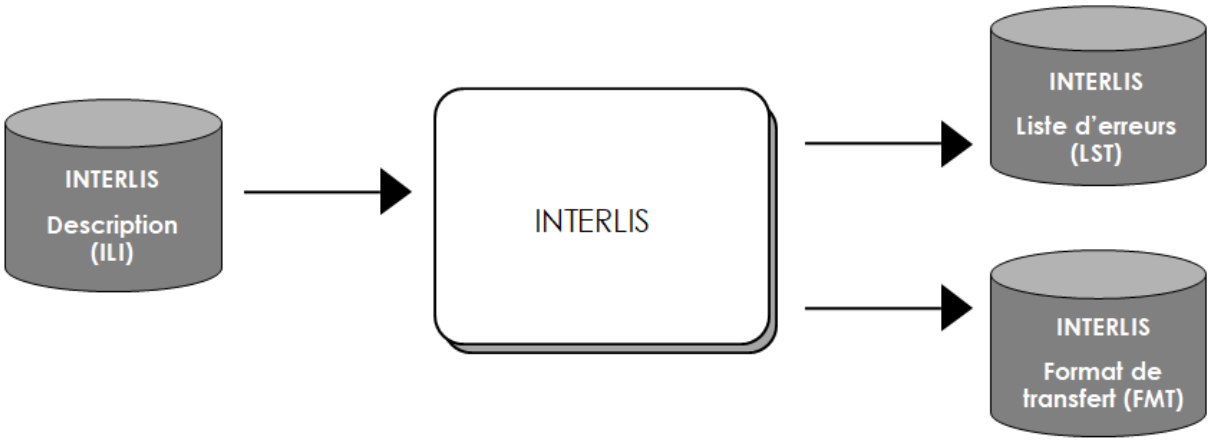


Figure 12 : Mécanisme de transfert d'un fichier Interlis 1

Exemple complet

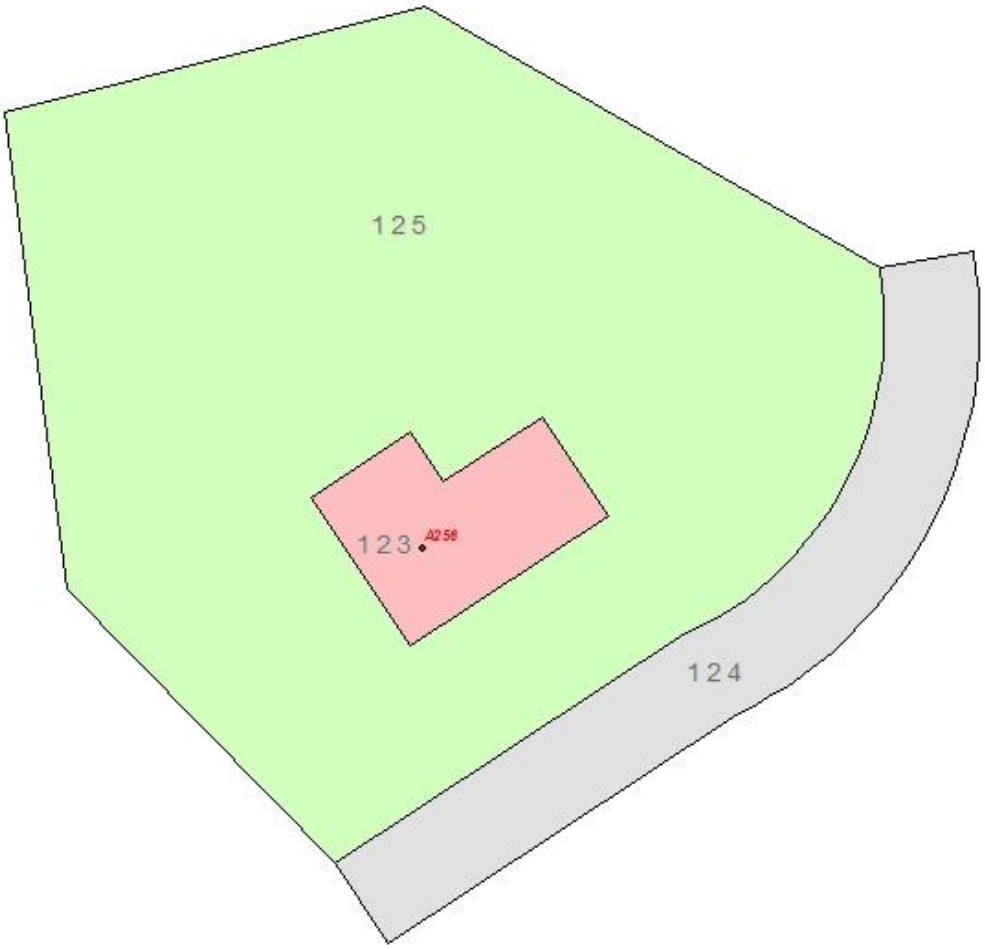


Figure 13 : Exemple de croquis

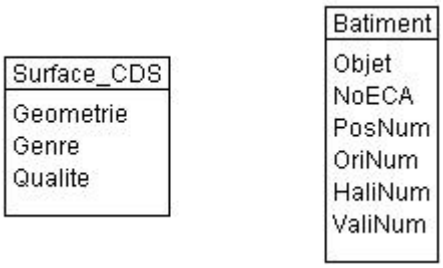


Figure 14 : Diagramme logique

Modèle de données *.ili

```
TRANSFER Test;

MODEL Exemple

DOMAIN
CoordN = COORD2 480000.00 60000.00
              850000.00 320000.00;

 TOPIC Couverture_du_sol =

  TABLE Surface_CDS =
   Geometrie : AREA WITH (STRAIGHTS, ARCS) VERTEX CoordN
               WITHOUT OVERLAPS > 0.10;
   Genre : (batiment, revetement_dur, verte, eau, boisee,
           sans_vegetation);
   Qualite : (MO93, NP);
  NO IDENT
  END Surface_CDS;

  TABLE Batiment =
   Objet : -> Surface_CDS // Genre = batiment //;
   NoECA : TEXT*6;
   PosNum : CoordN // position dans la surface //;
   OriNum : GRADS 0.0 399.9;
   HaliNum : HALIGNMENT;
   ValiNum : VALIGNMENT;
  IDENT
   NoECA; !! NoECA est univoque
  END Batiment;

 END Couverture_du_sol.

END Exemple.

FORMAT FREE;
CODE
BLANK = DEFAULT, UNDEFINED = DEFAULT, CONTINUE = DEFAULT;
TID = ANY;
END.
```

Format de transfert *.FMT

```
SCNT
Beschreibung der Transferdatei
////
MTID model.ili
MODL Exemple
DOMAIN

!!!!
TOPI Couverture_du_sol
!!!!
TABL Surface_CDS_Geometrie
OBJE 1
!!!!
!!!! 1: Objektidentifikation
!!!!
STPT  111111.11 111111.11
LIPT  111111.11 111111.11
ARCP  111111.11 111111.11
ELIN
ETAB
!!!!
TABL Surface_CDS
OBJE 1 222222.22 222222.22 3 4
!!!!
!!!! 1: Objektidentifikation
!!!! 2: Geometrie
!!!! 3: Genre
!!!! 4: Qualite
!!!!
ETAB
!!!!
TABL Batiment
OBJE 1 2 333333 444444.44 444444.44 555.5 6 7
!!!!
!!!! 1: Objektidentifikation
!!!! 2: Objet ->Surface_CDS
!!!! 3: NoECA
!!!! 4: PosNum
!!!! 5: OriNum
!!!! 6: HaliNum
!!!! 7: ValiNum
!!!!
ETAB
ETOP
EMOD
ENDE
```

Données *.itf

```
SCNT
Fichier de transfert de l'exemple
////
MTID model.ili
MODL Exemple
TOPI Couverture_du_sol
TABL Surface_CDS_Geometrie
OBJE 1
STPT    600146.92 200174.98
LIPT    600138.68 200187.51
LIPT    600147.04 200193.00
LIPT    600149.79 200188.82
LIPT    600158.15 200194.31
LIPT    600163.64 200185.96
LIPT    600146.92 200174.98
ELIN
OBJE 2
STPT    600140.69 200156.63
LIPT    600118.19 200179.82
LIPT    600113.00 200219.97
LIPT    600148.30 200228.97
LIPT    600186.38 200206.82
ELIN
OBJE 3
STPT    600186.38 200206.82
ARCP    600183.52 200188.65
LIPT    600170.17 200176.00
LIPT    600140.69 200156.63
ELIN
OBJE 4
STPT    600186.38 200206.82
LIPT    600194.26 200208.19
ARCP    600190.75 200185.21
LIPT    600174.10 200169.00
LIPT    600145.08 200149.94
LIPT    600140.69 200156.63
ELIN
ETAB
TABL Surface_CDS
OBJE 123 600148.20 200183.48 0 0
OBJE 124 600168.27 200170.85 1 0
OBJE 125 600133.95 200206.06 2 0
ETAB
TABL Batiment
OBJE 552 123 A256 600148.00 200183.20 0.0
0 4
ETAB
ETOP
EMOD
ENDE
```

4. Remarques concernant INTERLIS

4.1. Quelques règles

La liste ci-dessous n'est pas exhaustive !

Essayez de décrire un modèle correct en vue de l'application, sans tenir compte des éventuelles restrictions d'un système concret

Fixez uniquement les relations nécessaires

Groupez les entités d'un thème en un TOPIC

Des relations entre TOPIC's ne sont pas admises

Si les mêmes valeurs d'attributs existent à plusieurs endroits (dans un modèle ou dans un thème), utilisez la possibilité de définir des domaines

Si possible, utilisez des listes avec des valeurs d'attributs prédéfinies (énumérations)

Renoncez à définir un attribut, si la valeur est pour la plupart du temps inconnu et si la saisie et la mise à jour ne sont pas assurées du point de vue organisationnel et financier.

4.2. Concernant le style de description

Le style de décrire un modèle avec INTERLIS est une question de goût. Un « bon » style permet d'améliorer la lisibilité d'un document. Quel que soit le style utilisé, il est important de l'appliquer de manière homogène.

Voici quelques règles ou conseils :

- en principe il faut insérer une ligne vide avant les définitions telles que DOMAIN et TABLE. Avant TOPIC on devrait trouver deux lignes vides.
- après DOMAIN, TOPIC, TABLE et des définitions d'attributs on insère deux caractères vides (ou un tabulateur). Après TRANSFER, MODEL, DERIVATIVES, VIEWS et FORMAT on ne trouve donc pas d'alinéa.
- les attributs d'une table sont décrits l'un en dessous de l'autre et chacun sur une propre ligne.
- types et commentaires devraient être alignés dans les mêmes colonnes.
- pour obtenir des noms abrégés, des couches d'informations, des tables et des noms d'attributs peuvent contenir des points, comme par exemple : « Couvsol.Bati.Geometrie ».

5. Bibliographie et références

- Cours de géoinformatique, module 3B, Francis Grin, EIVD 2003
- Cours Interlis 1, CF-geo, Robert Balanche, octobre 2013
- Cours Interlis 1, CF-geo, Isabelle Pointet, octobre 2016
- Cours INTERLIS 1, swisstopo – EIVD, 2003
- Manuel de référence d'INTERLIS 1 - Mécanisme d'Echange de Données Pour Systèmes d'Information du territoire, Version 1, Révision 2, mars 1998
  - ➔ https://www.interlis.ch/download/interlis1/ili1_ref_f.pdf
- Modèle de données 2001 de la mensuration officielle "Confédération" (MD.01-MO-CH), swisstopo

6. Exercices - Interlis 1

6.1. Re-engineering

D'après la description en INTERLIS 1 ci-dessous, reconstituer le modèle conceptuel selon le formalisme „entité-relation"

```
TOPIC Altimetrie =

    TABLE Mise_a_jourAL =
      IdentDN: TEXT*12; !! relation 1-m avec Domaine_numerotation
      Identification: TEXT*12;  !! par ex. numero du dossier technique
      Description: TEXT*30;
      Perimetre: OPTIONAL SURFACE WITH (STRAIGHTS, ARCS) VERTEX CoordP
        WITHOUT OVERLAPS > 0.200;
      Validite: Statut;
      !! Pour les mises a jour futures la date a renseigner est En_vigueur.
      !! Date1 correspond aux anciennes mises a jour.
      !! Dans la prochaine revision du modele de donnees Date1 sera supprime et
      !! En_vigueur sera declare obligatoire.
      En_vigueur: OPTIONAL DATE;
      Date1: OPTIONAL DATE; !! par ex. date de mise en service
    IDENT IdentDN, Identification;
    END Mise_a_jourAL;

    TABLE Point_cote = !! valable ou projete
      Origine: -> Mise_a_jourAL; !! relation 1-mc
      Geometrie: CoordA;
      Qualite: StandardQualite;
      IDENT Geometrie;
    END Point_cote;

    TABLE PosPoint_cote =
      PosPoint_cote_de: -> Point_cote; !! relation 1-c; inscription de Geometrie
      Pos: CoordP;
      Ori: OPTIONAL Rotation // indefini = 100.0 //;
      HAli: OPTIONAL HALIGNMENT // indefini = Left //;
      VAli: OPTIONAL VALIGNMENT // indefini = Bottom //;
    IDENT PosPoint_cote_de;
    END PosPoint_cote;

    TABLE Arete =
      Origine: -> Mise_a_jourAL; !! relation 1-mc
      Geometrie: POLYLINE WITH (STRAIGHTS) VERTEX CoordA; !! sans ARCS !
      Qualite: StandardQualite;
      Genre: (
        ligne_de_rupture,
        ligne_de_structure,
        autre); !! Pas d'objet dans la categorie autre, uniquement pour extensions
    NO IDENT
    END Arete;

    TABLE Surface_vide =
      Origine: -> Mise_a_jourAL;  !! relation 1-mc
      Geometrie: SURFACE WITH (STRAIGHTS, ARCS) VERTEX CoordP;
      Qualite: StandardQualite;
      Genre: (
        surface_morte,
        autre); !! Pas d'objet dans la categorie autre, uniquement pour extensions
    NO IDENT
    END Surface_vide;
  END Altimetrie.
```

6.2. Description du modèle « Parcelle / Pos_Numero » en modèle de données INTERLIS 1

Décrire le modèle ci-dessous en modèle de données INTERLIS 1 (fichier *.ili), selon cadre de référence suisse MN03.

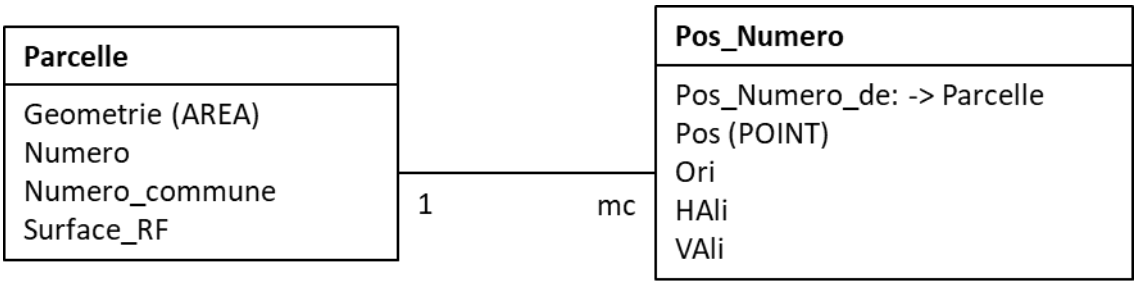


Figure 15 : Modèle « Parcelle / Pos_Numero »

Remarques :

- L'identifiant de la table « Parcelle » est « Numero »
- La table « Pos_Numero » n'a pas d'identifiant

6.3. Description du modèle « Batiment / Logement » en modèle de données INTERLIS 1

Décrire le modèle ci-dessous en modèle de données INTERLIS 1 (fichier *.ili), selon cadre de référence suisse MN95.

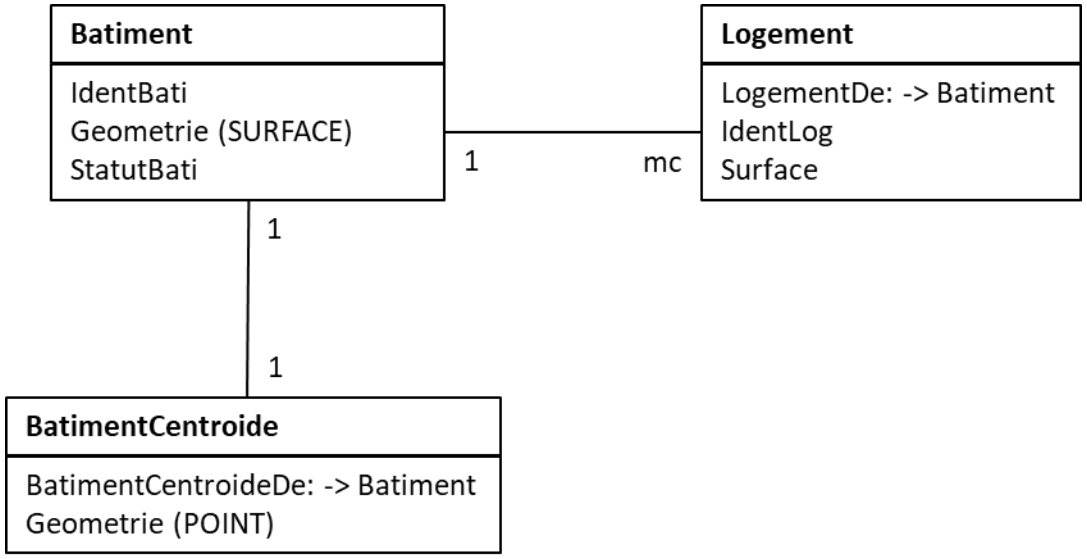


Figure 16 : Modèle « Batiment / Logement »

Remarques :

- L'attribut « StatutBati » de la table « Batiment » correspond à un domaine de valeurs (« DOMAIN ») avec les valeurs possibles :
  - « En_Projet »,
  - « Demoli »,
  - « Construit ».
- L'identifiant de la table « Batiment » est « IdentBati »,
- L'identifiant de la table « BatimentCentroide » est « BatimentCentroideDe »,
- L'identifiant de la table « Logement » est une clé composée avec les champs « LogementDe », « IdentLog ».

6.4. Exercice sur un jeu de données ITF

A l'aide des fichiers **md01f.ili** et du jeu de données test **md01f.itf**, répondre aux questions ci-dessous.

1. TOPIC Points_fixesCategorie2, indiquer l'altitude du point PFA2 ZH302202 / 57 :

2. TOPIC Points_fixesCategorie3, indiquer la description de la mutation concernant le point PFP3 numéro : ZH302202 / 87 :

3. TOPIC Points_fixesCategorie3, indiquer les coordonnées du point PFP3 ZH302201 / 81 et les coordonnées de la position du numéro du point :

4. TOPIC Couverture du sol, dans la table SurfaceCS, indiquez le genre de couverture du sol des objets dont l'OBJECT ID vaut 0, 1 et 2 :

5. TOPIC Couverture du sol, dans la table SurfaceCS, considérons les objets ayant un OBJECT ID = 21 et 22, que signifie la valeur 20 ? A quoi se rattache cette valeur ?

6. TOPIC Couverture du sol, dans la table SurfaceCS, a quoi correspond la paire de coordonnées indiquée ?

7. TOPIC Couverture du sol, table Numero_de_batiment, considérons l'objet ayant un OBJECT ID = 4. Indiquez le genre de couverture du sol et la signification du dernier terme :

8. TOPIC Objets_divers, table Element_surfacique, donnez la géométrie de l'objet ayant un OBECT ID = 1988 et le genre d'objet divers considéré :

9. TOPIC Objets_divers, table Point_particulier, donnez la description de la mutation des objets ayant l'OBJECT ID = 17500 et 17788 :

10. TOPIC Conduites, table Element_lineaire, combien d'objets avons-nous dans cette table ?

11. TOPIC Conduites, table Point_particulier, pour le point ayant une identification = 12351, indiquez la position, l'orientation et les alignements de l'inscription du numéro :

6.5. Description du modèle « Plan des zones » en modèle de données INTERLIS 1

Retranscrire le modèle de données « Plan des zones » ci-dessous en modèle de données INTERLIS 1 (fichier *.ili).

Vérifier la syntaxe de la description avec le compilateur INTERLIS.

Si le résultat du compilateur INTERLIS présente des erreurs, corriger la syntaxe du fichier *.ili jusqu'à ce qu'il soit propre.

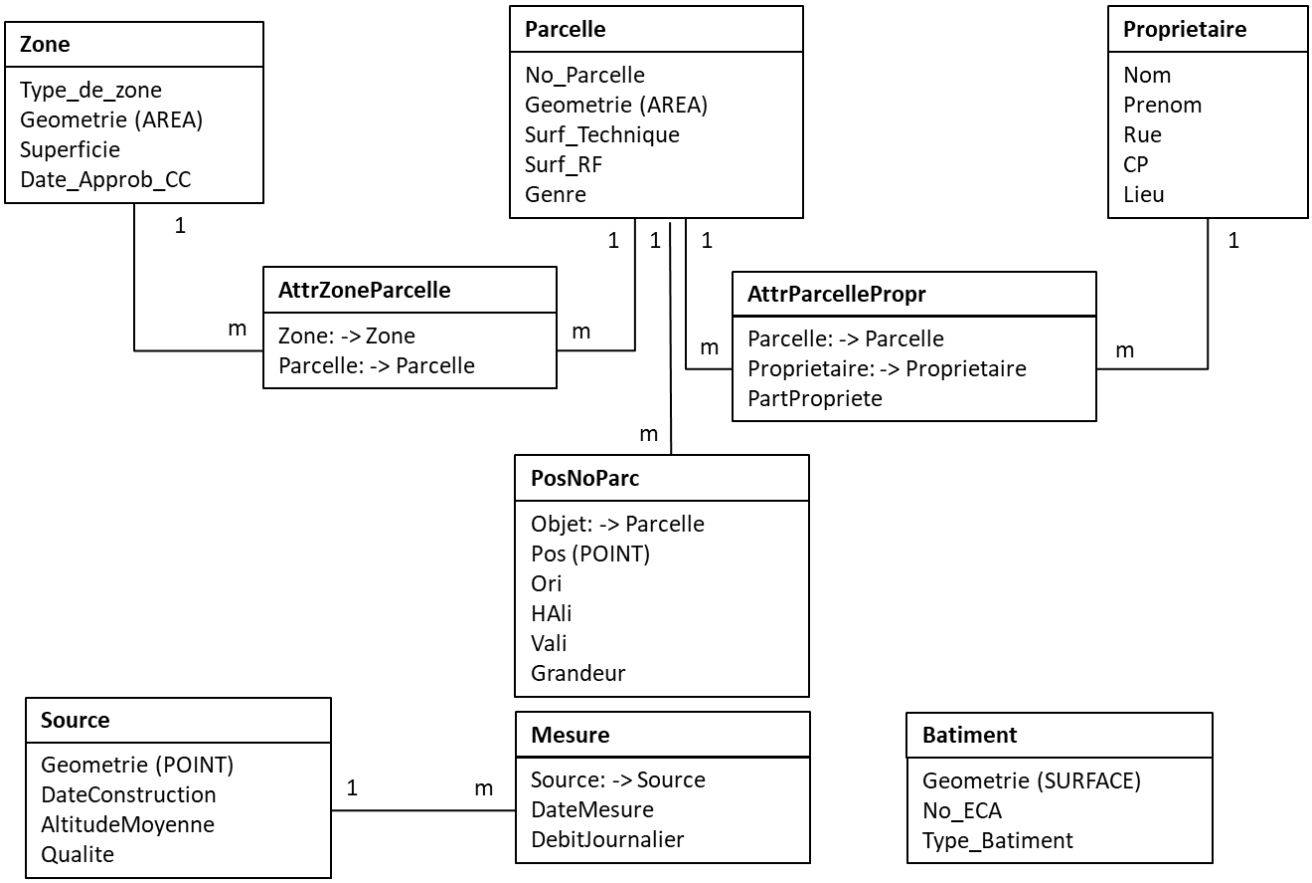


Figure 17 : Modèle « Plan des zones »

Veuillez consulter les remarques énoncées dans les deux prochaines pages.

Remarques :

- L'attribut « Genre » de la table « Parcelle » correspond à un domaine de valeurs (« DOMAIN ») avec les valeurs possibles :

  - prive,
  - dp (cantonal, communal).

- L'attribut « Grandeur » de la table « PosNoParc » correspond à un domaine de valeurs (« DOMAIN ») avec les valeurs possibles :

  - petite,
  - moyenne,
  - grande.

- L'attribut « Type_Batiment » de la table « Batiment » correspond à un domaine de valeurs (« DOMAIN ») avec les valeurs possibles :

  - habitation,
  - commerce,
  - usine,
  - porcherie,
  - Autre.

- L'attribut « Type_de_zone » de la table « Zone » correspond à un domaine de valeurs (« DOMAIN ») avec les valeurs possibles :

  - agricole,
  - agricole_protegee,
  - attente_1,
  - attente_2,
  - attente_4,
  - interdiction_batir,
  - utilite_publique,
  - protection,
  - reserve_naturelle_protegee,
  - verdure,
  - village,
  - intermediaire,
  - peripherique_a,
  - peripherique_c,
  - peripherique_d,
  - peripherique_e,
  - ppa,
  - protegee,
  - Viticole.

Remarques :

- L'attribut « AltitudeMoyenne » de la table « Source » est compris entre -200.000 et 5'000.00 mètres,
- L'attribut « DebitJournalier » de la table « Mesure » est compris entre 0.00 et 1'000.00 (l/m),
- L'attribut « Ori » de la table « PosNoParc » est en grades (GRADS) et varie de 0.0 à 399.9,
- L'attribut « PartPropriete » de la table « AttrParcellePropr » est compris entre 0.00 et 100.00 (%),
- L'attribut « Qualite » de la table « Source » est compris entre 1 et 10,
- Les attributs « Superficie » et « Surf_RF » des tables « Zone » et « Parcelle » respectivement varient de 0 à 1'000'000 m^2,
- L'attribut « Surf_Technique » de la table « Parcelle » varie de 0.00 à 1'000'000.00 m^2,
- L'identifiant de la table « Batiment » est « No_ECA »,
- L'identifiant de la table « Parcelle » est « No_Parcelle »,
- L'identifiant de la table « Source » est « Geometrie »,
- Les tables « AttrParcelleProp », « AttrZoneParcelle », « PosNoParc », « Proprietaire », « Source » et « Zone » n'ont pas d'identifiant.

6.6. Identification d'erreurs dans un jeu de données ITF

1 - A l'aide de l'outil ILIVALIDATOR et du modèle de données **NORMAT.ili**, contrôler le jeu de données stocké dans le fichier **donneestest.itf**, créant un fichier résultat nommé **donneestest.log**.

Identifier et décrire les erreurs contenues dans le jeu de données stocké dans le fichier **donneestest.itf**.

Il s'agit de décrire par classe d'objet chaque erreur relevée par l'outil ILIVALIDATOR, notamment :

- l'identifiant de l'objet en erreur
- l'attribut concerné (nom de la colonne défini dans le modèle et numéro de la colonne dans le fichier de données)
- la valeur en erreur
- la valeur attendue et autorisée

2 – Considérant le modèle de données **NORMAT.ili**, corriger le jeu de données stocké dans le fichier **donneestest.itf** pour ne plus avoir d'erreurs lors du contrôle fait via l'outil ILIVALIDATOR.